



# UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS  
PROFESIONALES  
ZONA MEDIA

INGENIERÍA MECATRÓNICA



## PRÁCTICA 7

### Contador Binario Ascendente de 4 Bits Mediante Pulsador en RPi Pico W

**MATERIA:** MICROCONTROLADORES

**DOCENTE:** Ing. JESÚS PADRÓN

**ALUMNO:** CARLOS ANGEL GARCÍA SIFUENTES

**CLAVE:** A383917

**CARRERA:** INGENIERÍA MECATRÓNICA

**SEMESTRE:** 4° SEMESTRE

**FECHA:** 20 DE MAYO DE 2026

# Contador Binario Ascendente de 4 Bits con Resistencia

## Pull-Down Interna y Anti-Rebote en Raspberry Pi Pico W

CARLOS ÁNGEL GARCÍA SIFUENTES

a383917@alumnos.uaslp.mx

Fecha: 20 de mayo de 2026

**Resumen**—Se presenta el desarrollo de la Práctica 7, cuyo objetivo es implementar un contador binario ascendente de 4 bits en la Raspberry Pi Pico W, controlado por un pulsador y visualizado mediante cuatro LEDs externos. El programa fue escrito en lenguaje C utilizando el SDK oficial de Raspberry Pi, aprovechando la resistencia interna de configuración *pull-down* del microcontrolador para simplificar el circuito respecto a la práctica anterior. Se implementó además un algoritmo de anti-rebote (*debounce*) por software que detecta el flanco de subida y espera la liberación del botón antes de incrementar el contador, garantizando una pulsación limpia por cada presión. El contador avanza de 0 a 15 en binario y regresa automáticamente a 0 al desbordarse.

## I. INTRODUCCIÓN

### I-A Descripción General de la Práctica

En esta práctica se construye un sistema digital básico que combina lectura de entrada digital con escritura en múltiples salidas simultáneas. Cada pulsación del botón incrementa en una unidad un contador de 4 bits, cuyo valor se refleja en tiempo real sobre cuatro LEDs conectados a los pines GPIO 2, GPIO 3, GPIO 4 y GPIO 5, representando respectivamente el bit 0 (LSB) hasta el bit 3 (MSB) de la representación binaria. Al alcanzar el valor 15 (1111 en binario), la siguiente pulsación reinicia el contador a 0 (0000), generando un ciclo continuo.

### I-B Trabajo Previo: Función `gpio_set_pulls()`

Como trabajo previo se analizó la función `gpio_set_pulls()` documentada en el capítulo 4 del manual *Raspberry Pi Pico-series C/C++ SDK* (sección 4.1.10.7.65). Esta función configura las resistencias internas de *pull-up* o *pull-down* de un pin GPIO sin necesidad de componentes externos. Su prototipo es:

**PROTOTIPO — `gpio_set_pulls()`**

```
void gpio_set_pulls(uint gpio, bool up,
```

```
bool down);
```

Los parámetros son los siguientes: `gpio` es el número del pin a configurar; `up` habilita la resistencia interna de *pull-up* cuando es `true`; y `down` habilita la resistencia interna de *pull-down* cuando es `true`. Ambos parámetros pueden coexistir, aunque en la práctica se activa únicamente uno a la vez para definir el estado en reposo del pin de forma inequívoca.

En esta práctica se invoca como `gpio_set_pulls(BUTTON, false, true)`, activando la resistencia *pull-down* interna y dejando el pin en nivel lógico bajo (0) cuando el botón está sin presionar, lo que elimina la necesidad de la resistencia física de 10 kΩ utilizada en la práctica anterior.

### I-C Representación Binaria de 4 Bits

Un número de 4 bits puede representar valores enteros del 0 al 15. La Tabla I muestra la correspondencia entre el valor decimal del contador y los estados de los cuatro LEDs.

Tabla I: Tabla de conteo binario de 4 bits (0–15)

| Decimal | BIT3 (MSB) | BIT2 | BIT1 | BIT0 (LSB) |
|---------|------------|------|------|------------|
| 0       | 0          | 0    | 0    | 0          |
| 1       | 0          | 0    | 0    | 1          |
| 2       | 0          | 0    | 1    | 0          |
| 3       | 0          | 0    | 1    | 1          |
| 4       | 0          | 1    | 0    | 0          |
| 5       | 0          | 1    | 0    | 1          |
| 6       | 0          | 1    | 1    | 0          |
| 7       | 0          | 1    | 1    | 1          |
| 8       | 1          | 0    | 0    | 0          |
| 9       | 1          | 0    | 0    | 1          |
| 10      | 1          | 0    | 1    | 0          |
| 11      | 1          | 0    | 1    | 1          |
| 12      | 1          | 1    | 0    | 0          |
| 13      | 1          | 1    | 0    | 1          |
| 14      | 1          | 1    | 1    | 0          |
| 15      | 1          | 1    | 1    | 1          |

## II. DESARROLLO

### II-A Material y Recursos Utilizados

Para el correcto armado y ejecución de la práctica se emplearon los siguientes componentes:

- **Hardware:** 1 Placa Raspberry Pi Pico W, 1 cable micro USB tipo B, 4 LEDs verdes, 1 fuente de alimentación externa (3.3 V–5.5 V), 4 resistencias de 45  $\Omega$  (calculadas para LED verde con una corriente de 20 mA), y 1 pulsador sin enclavamiento mecánico.
- **Software y librerías:** SDK de Raspberry Pi Pico, entorno Pico – Visual Studio Code, y la biblioteca `pico_stdlib`.

### II-B Estructura del Proyecto y Configuración de CMake

Se acondicionó la carpeta de proyecto `PRACTICA_7` con el script `pico_sdk_import.cmake` y los subdirectorios `.vscode`. El archivo `CMakeLists.txt` utilizado es el siguiente:

#### ARCHIVO CMakeLists.txt

```
cmake_minimum_required(VERSION 3.13)

include(pico_sdk_import.cmake)
set(PICO_BOARD pico_w)

project(PRACTICA_7)

pico_sdk_init()

add_executable(Contador
    Contador.c
)

target_link_libraries(Contador
    pico_stdlib
)

pico_add_extra_outputs(Contador)
```

### II-C Implementación de la Lógica de Control en C

El archivo `Contador.c` define cinco macros para los pines, una variable global de 8 bits para el conteo, y una función auxiliar `actualizar_gpio()` que descompone el valor del contador en sus cuatro bits individuales y los escribe en los pines de salida correspondientes.

#### CÓDIGO FUENTE – Contador.c

```
#include "pico/stdlib.h"

// Definicion de pines
#define BIT0 2 // LSB
#define BIT1 3
#define BIT2 4
#define BIT3 5 // MSB
#define BUTTON 10

// Variable para almacenar el conteo
uint8_t contador = 0;

// Funcion para actualizar las salidas GPIO
void actualizar_gpio(uint8_t valor){
    gpio_put(BIT0, valor & 0x01);
    gpio_put(BIT1, (valor >> 1) & 0x01);
    gpio_put(BIT2, (valor >> 2) & 0x01);
    gpio_put(BIT3, (valor >> 3) & 0x01);
}

int main(){

    gpio_init(BIT0);
    gpio_init(BIT1);
    gpio_init(BIT2);
    gpio_init(BIT3);
    gpio_set_dir(BIT0, GPIO_OUT);
    gpio_set_dir(BIT1, GPIO_OUT);
    gpio_set_dir(BIT2, GPIO_OUT);
    gpio_set_dir(BIT3, GPIO_OUT);

    gpio_init(BUTTON);
    gpio_set_dir(BUTTON, GPIO_IN);
    gpio_set_pulls(BUTTON, false, true); // Pull-down interno

    actualizar_gpio(contador); // Estado inicial: 0000

    while(1){
        if (gpio_get(BUTTON) == 1){
            sleep_ms(50);

            // Anti-rebote subida
            while (gpio_get(BUTTON) == 1)
                sleep_ms(50);

            // Esperar soltar boton

            contador = (contador + 1) & 0x0F; // Incremento ciclico
            actualizar_gpio(contador);
        }
    }
}
```

## II-C1. Análisis de la función `actualizar_gpio()`

La función `actualizar_gpio(uint8_t valor)` recibe el valor actual del contador (un entero de 0 a 15) y lo descompone en sus cuatro bits para escribirlos individualmente en cada pin de salida:

- `gpio_put(BIT0, valor & 0x01)`: extrae el bit 0 aplicando una máscara AND con 0x01 (0000 0001 en binario).
- `gpio_put(BIT1, (valor >> 1) & 0x01)`: desplaza el valor un bit a la derecha y aplica la misma máscara, extrayendo el bit 1.

`gpio_put (BIT2, (valor & 0x01):` desplaza dos posiciones para extraer el bit 2.

gpio\_put(BIT3, (valor & 0x01): desplaza tres posiciones para extraer el bit 3 (MSB).

De esta forma, los cuatro LEDs reflejan en todo momento la representación binaria exacta del contador.

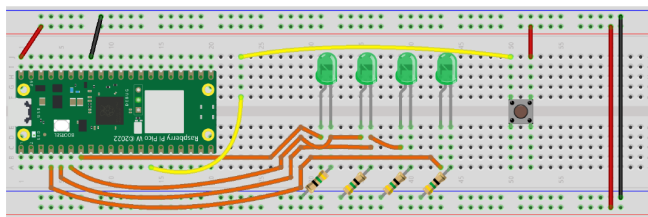


Figura 1: Diagrama del circuito en protoboard: 4 LEDs conectados a GPIO 2–5 con resistencias de 45  $\Omega$ , pulsador en GPIO 10 con *pull-down* interno y alimentación externa en rieles laterales.

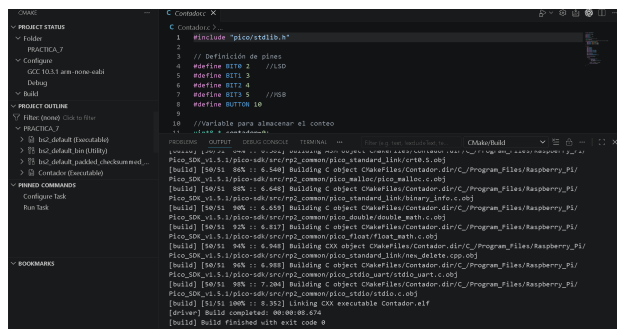


Figura 2: Compilación exitosa del proyecto en Pico Visual Studio Code (exit code 0).

## II-C2. Lógica de anti-rebote y detección de flanco

El bloque `while(1)` implementa la siguiente secuencia ante cada pulsación:

1. Detecta el flanco de subida (`gpio_get (BUTTON) == 1`).
2. Espera 50 ms para que el rebote mecánico se estabilice.
3. Permanece en espera activa mientras el botón siga presionado (`while gpio_get == 1`), garantizando que solo se incrementa **una vez** por pulsación completa.
4. Espera 50 ms adicionales al soltar, cubriendo el rebote de apertura.
5. Incrementa el contador y lo limita a 4 bits con la máscara `& 0x0F`.

## II-D Conexión del Circuito

El circuito se armó sobre una protoboard conforme al diagrama de la Figura 1. A diferencia de la práctica anterior, no se requiere resistencia física en el pulsador gracias a la resistencia *pull-down* interna activada por software. El pin 40 de la Pico W (VBUS/3V3) suministra 3.3 V al riel positivo de la protoboard para alimentar los LEDs.

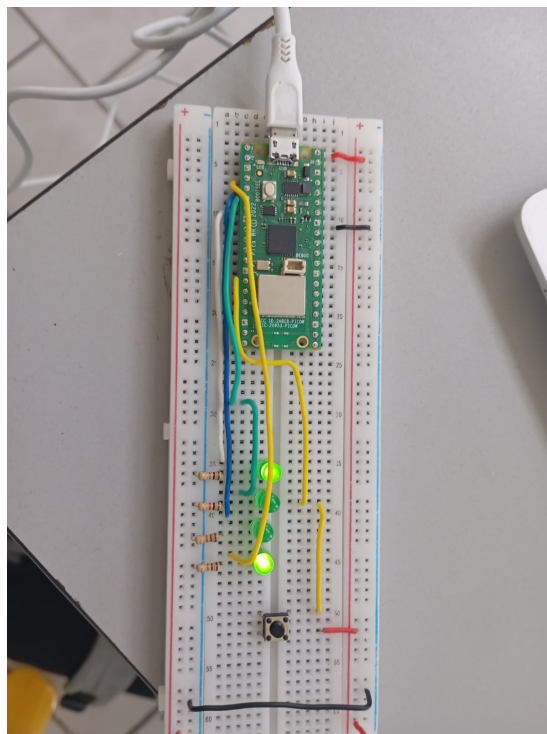


Figura 3: Circuito físico en funcionamiento mostrando un valor binario en los cuatro LEDs.

### III. PREGUNTAS DE REPASO

**III-A** ¿Qué pasa si `gpio_set_pulls(BUTTON, false, true)` cambia por `gpio_set_pulls(BUTTON, true, false)`?

Al intercambiar los parámetros se desactiva la resistencia *pull-down* interna y se activa en su lugar la resistencia *pull-up* interna. Esto invierte completamente la lógica del pin: en reposo (botón sin presionar), el pin GPIO10 quedaría conectado internamente a 3.3V, leyendo permanentemente un nivel alto (1 lógico). Al presionar el botón y conectar el pin a GND, el nivel bajaría a 0 lógico.

Como consecuencia, el programa detectaría el botón como *siempre presionado* en reposo y correría el contador indefinidamente sin que el usuario toque nada. Para que el código funcione correctamente con esta configuración *pull-up*, habría que invertir la condición de detección a `gpio_get(BUTTON) == 0` y conectar el otro extremo del pulsador a GND en lugar de a 3.3V.

**III-B** ¿Cuál es la función del cable rojo que sale del pin 40 de la RPi Pico W al riel positivo de la protoboard?

El pin físico 40 de la Raspberry Pi Pico W corresponde al pin **VBUS**, entregando el voltaje proveniente del bus USB (nominalmente 5V) cuando la placa está conectada por cable. Pero, en el diagrama de la práctica se usa para llevar la tensión regulada de 3.3V de la placa hacia el riel positivo de la protoboard, de modo que los LEDs y el pulsador tengan una referencia de alimentación común y consistente con los niveles lógicos del microcontrolador. Esto permite que todo el circuito comparta la misma referencia de voltaje que los pines GPIO, evitando niveles incompatibles que podrían dañar el RP2040.

**III-C** Explica el bloque de código de las líneas 14 a 19 del archivo `Contador.c`

El bloque en cuestión es la función `actualizar_gpio(uint8_t valor):`

#### BLOQUE ANALIZADO — líneas 14 a 19

```
void actualizar_gpio(uint8_t valor){
    gpio_put(BIT0, valor & 0x01);
    gpio_put(BIT1, (valor >> 1) & 0x01);
    gpio_put(BIT2, (valor >> 2) & 0x01);
    gpio_put(BIT3, (valor >> 3) & 0x01);
}
```

Esta función recibe como parámetro el valor actual del contador (un número entre 0 y 15) y se encarga de escribir cada uno de sus 4 bits en el pin GPIO correspondiente. Lo hace mediante dos operaciones bit a bit combinadas:

- El operador de desplazamiento a la derecha (`>>`) mueve el bit de interés hasta la posición 0. Por ejemplo, `valor >> 2` coloca el bit 2 en la posición del LSB.
- El operador AND con la máscara `0x01` (es decir, `0000 0001`) aísla únicamente ese bit, descartando los demás.

El resultado es que `gpio_put()` recibe exactamente un 0 o un 1, encendiendo o apagando el LED correspondiente para reflejar fielmente el valor binario del contador en los cuatro LEDs.

### IV. CONCLUSIONES

- El uso de la resistencia interna *pull-down* mediante `gpio_set_pulls()` simplificó notablemente el circuito físico respecto a la práctica anterior, eliminando la necesidad de una resistencia de 10 kΩ externa y reduciendo el número de componentes sin sacrificar la estabilidad de la lectura digital.
- La función auxiliar `actualizar_gpio()` demostró la utilidad de encapsular operaciones repetitivas en funciones propias, haciendo el código más legible, mantenible y reutilizable, ya que cualquier cambio en la asignación de pines solo requiere modificar las macros `BIT0-BIT3`.
- El algoritmo de anti-rebote garantizó una pulsación limpia por cada presión física del botón, combinando un retardo de subida y una espera activa hasta la liberación, lo que es una técnica robusta y de bajo costo computacional para sistemas embebidos sin sistema operativo.
- La operación de enmascaramiento `(contador + 1) & 0x0F` resultó ser una forma eficiente de limitar el contador a 4 bits con desbordamiento automático a cero, prescindiendo de comparaciones adicionales y aprovechando la aritmética binaria del procesador.
- Como mejora futura se podría implementar el contador mediante interrupciones de GPIO en lugar del lazo de sondeo (*polling*), lo que permitiría que el microcontrolador ejecute otras tareas en paralelo mientras espera la pulsación, mejorando la eficiencia del sistema.

## ANEXOS

### A. Código fuente – Contador.c

#### ANEXO A – Contador.c (completo)

```
#include "pico/stdlib.h"

#define BIT0 2
#define BIT1 3
#define BIT2 4
#define BIT3 5
#define BUTTON 10

uint8_t contador = 0;

void actualizar_gpio(uint8_t valor){
    gpio_put(BIT0, valor & 0x01);
    gpio_put(BIT1, (valor >> 1) & 0x01);
    gpio_put(BIT2, (valor >> 2) & 0x01);
    gpio_put(BIT3, (valor >> 3) & 0x01);
}

int main(){
    gpio_init(BIT0);    gpio_set_dir(BIT0,
    GPIO_OUT);
    gpio_init(BIT1);    gpio_set_dir(BIT1,
    GPIO_OUT);
    gpio_init(BIT2);    gpio_set_dir(BIT2,
    GPIO_OUT);
    gpio_init(BIT3);    gpio_set_dir(BIT3,
    GPIO_OUT);

    gpio_init(BUTTON);
    gpio_set_dir(BUTTON, GPIO_IN);
    gpio_set_pulls(BUTTON, false, true);

    actualizar_gpio(contador);

    while(1){
        if (gpio_get(BUTTON) == 1){
            sleep_ms(50);
            while (gpio_get(BUTTON) == 1)
                sleep_ms(50);
            contador = (contador + 1) & 0
                x0F;
            actualizar_gpio(contador);
        }
    }
}
```

### B. Archivo CMakeLists.txt

#### ANEXO B – CMakeLists.txt (completo)

```
cmake_minimum_required(VERSION 3.13)

include(pico_sdk_import.cmake)
set(PICO_BOARD pico_w)

project(PRACTICA_7)

pico_sdk_init()

add_executable(Contador
    Contador.c
)

target_link_libraries(Contador
    pico_stdlib
)

pico_add_extra_outputs(Contador)
```

### Fotografía del armado

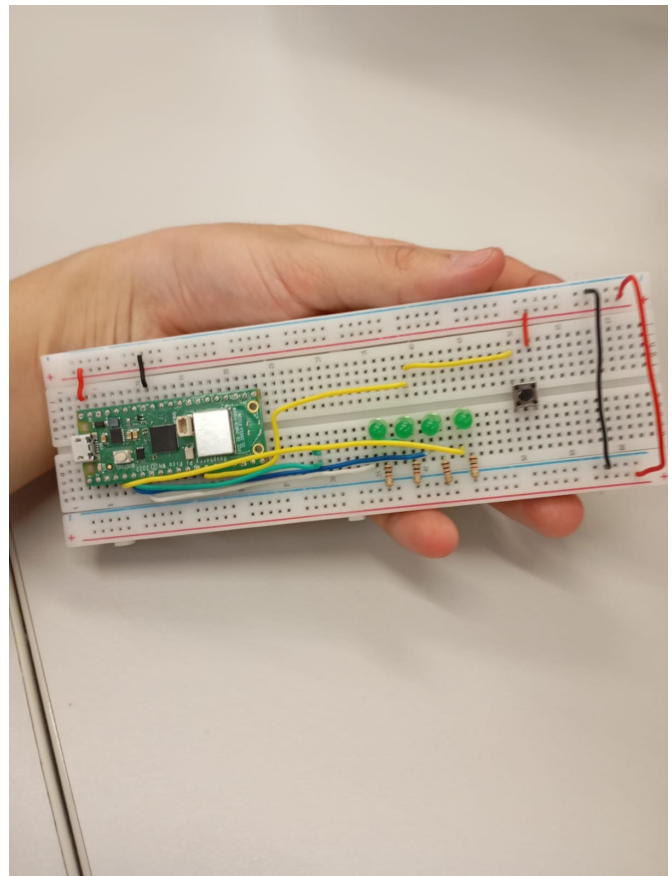


Figura 4: Fotografía del protoboard del contador binario de 4 bits (Fritzing).